# Software-In-the-Loop (SIL) & Serial Port Calculator

https://github.com/BabakHajari/Software-in-the-Loop-SIL-Modbus-RTU-Serial-Port

**The-Software-In-the-Loop (SIL) and Serial Port Calculator is a software tool designed for PLC programmers and engineers who require a simulation environment with serial communication ports (RS-232/RS-485)** to respond on behalf of real devices. It provides practical utilities for industrial communication, including:

- **Hex-to-decimal and decimal-to-hex conversion**
- **2-byte and 4-byte IEEE (floating-point) value calculation and conversion**
- **Hex-to-decimal and decimal-to-hex conversion**
- **Hex-to-string and string-to-hex conversion**
- **Simulate a machine and respond to preset serial port command** (SIL)
- Support for **Modbus protocol**

## Overview of the Modbus Protocol

**Modbus** is a widely used **industrial communication protocol** originally developed by **Modicon (now Schneider Electric)** in 1979. It is designed for reliable communication between **industrial electronic devices**, such as **PLCs, sensors, actuators, HMIs, and SCADA systems**.

Modbus is valued for its **simplicity, openness, and robustness**, making it one of the most commonly implemented protocols in industrial automation.

## Modbus Communication Model

Modbus follows a **master–slave** (also referred to as **client–server**) architecture:

- The **Master (Client)** initiates all communication requests.
- The **Slave (Server)** devices respond to the master's requests.
- Slave devices never communicate with each other directly.

Each slave device has a **unique address**, allowing the master to communicate with multiple devices on the same network.

## Modbus RTU

- Uses **binary data format**
- Communicates over **serial links** such as RS-485 or RS-232
- High efficiency and compact data frames
- Widely used in industrial environments

## Modbus Data Model

Modbus organizes data into four basic data types:

- **Coils** (Read/Write – Digital Outputs)
- **Discrete Inputs** (Read-only – Digital Inputs)
- **Input Registers** (Read-only – Analog Inputs)
- **Holding Registers** (Read/Write – Analog Values and Parameters)

Each data type is accessed using specific **function codes** defined by the Modbus standard.

## Modbus RTU Message Frame

A standard **Modbus RTU** frame consists of the following fields:

| Address | Function Code | Data | CRC |

## Field Descriptions

## 1. Address (1 byte)

- Identifies the target slave device
- Valid range: **1–247**
- Address **0** is reserved for broadcast messages (no response)

## 2. Function Code (1 byte)

Defines the action requested by the master.

| Function Code | Description |
|---|---|
| 01 | Read Coils |
| 02 | Read Discrete Inputs |
| 03 | Read Holding Registers |
| 04 | Read Input Registers |
| 05 | Write Single Coil |
| 06 | Write Single Register |
| 15 (0x0F) | Write Multiple Coils |
| 16 (0x10) | Write Multiple Registers |

## 3. Data Field (N bytes)

Contains parameters or returned values, depending on the function code.

## 4. CRC (2 bytes)

- Cyclic Redundancy Check for error detection
- Sent as **low byte first, then high byte**

## Example:

This example is a real Modbus message from the ADAM-4017+ module manufactured by Advantech. The module is configured to operate using the Modbus protocol.

Send → 10 04 00 00 00 08 F2 8D

Message Structure:

1. **Slave ID:** Identifies the target device.

   The first hexadecimal byte is 10, which represents the slave (node) address in hexadecimal (**17 in decimal**).

2. **Function Code:** Specifies the operation requested by the master (e.g., read or write).

   The second hexadecimal byte is 04, indicating the **Read Input Registers** function.

3. **Data**: Defines the registers to be accessed.
   - The third and fourth hexadecimal bytes (00 00) specify the **starting register address**.
   - The fifth and sixth hexadecimal bytes (00 08) indicate the **number of registers to be read**.

4. **CRC (Cyclic Redundancy Check)**: Used for error detection. The seventh and eighth hexadecimal bytes (F2 8D) represent the **CRC value** of the message.

Received → 10 03 10 7F FF 80 02 7F FF 7F FF 7F FC 7F FE 7F FE 7F FC 54 82

Message Structure:

1. **Slave ID:** Identifies the responding device

   The first hexadecimal byte is 10, which represents the slave (node) address in hexadecimal (**17 in decimal**).

2. **Function Code:** Specifies the operation performed.

   The second hexadecimal byte is 03, indicating the **Read Holding Registers** function.

3. **Data**: Contains the returned register values.
   - The third hexadecimal byte (10) specifies the **byte count**, indicating **16 bytes** of data follow.
   - The fourth through nineteenth hexadecimal bytes (7F FF 80 02 7F FF 7F FF 7F FC 7F FE 7F FE 7F FC) represent **eight 2-byte register values** returned by the slave.

4. **CRC (Cyclic Redundancy Check)**: Used for error detection. The twentieth and twenty-first hexadecimal bytes (54 82) represent the **CRC value** of the message.

The message structure for other Modbus devices may be similar; however, there are differences in the details, including the representation of real data types, which typically use IEEE floating-point format.

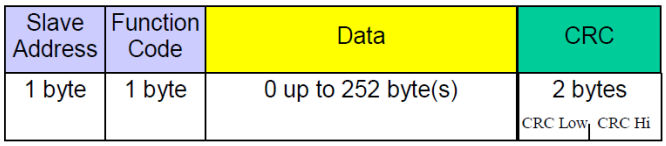
## Exception Response Format

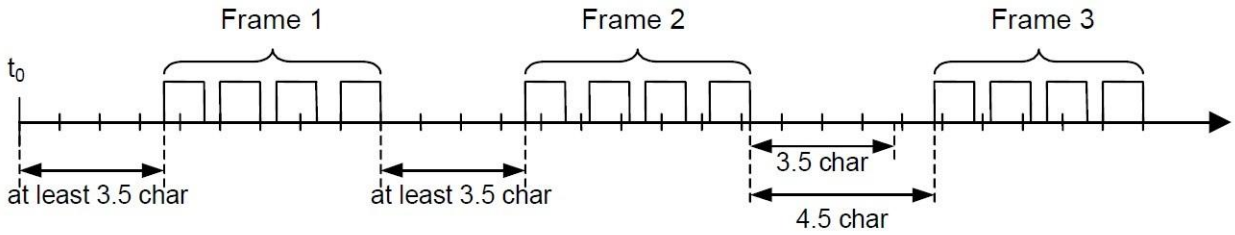
If an error occurs, the slave responds as follows:

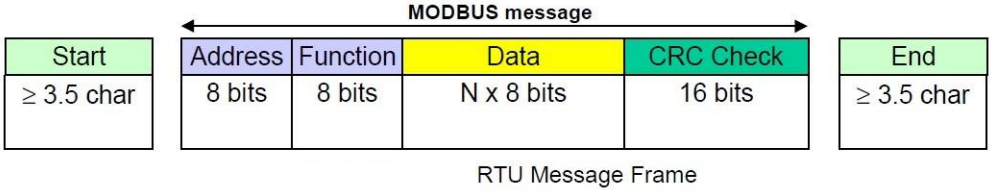
| Address | Function Code + 0x80 | Exception Code | CRC |

**Common Exception Codes:**

- 01 – Illegal Function
- 02 – Illegal Data Address
- 03 – Illegal Data Value
- 04 – Slave Device Failure



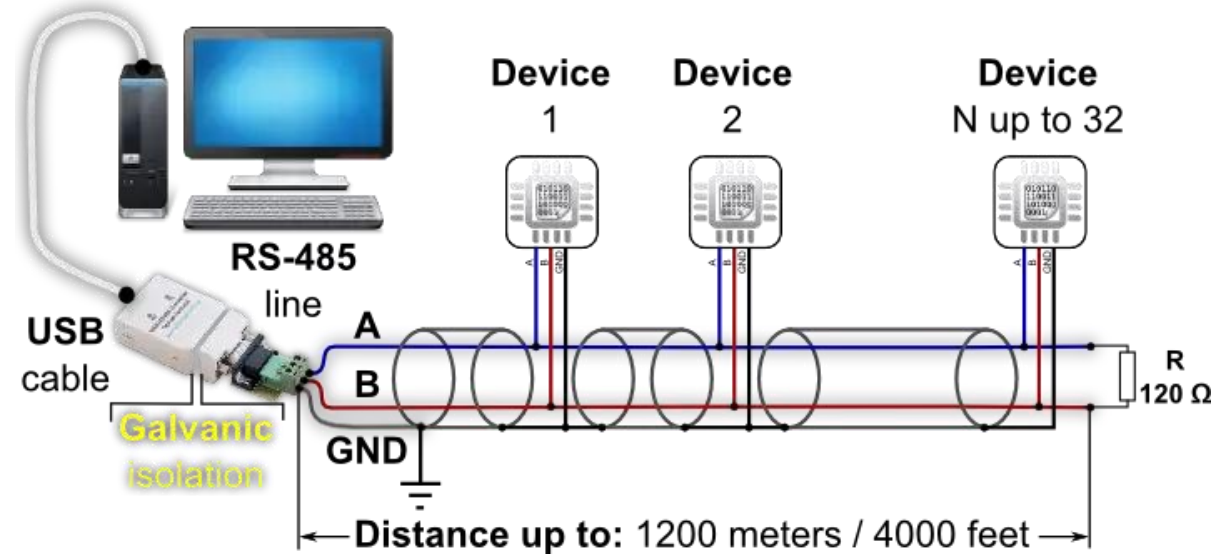



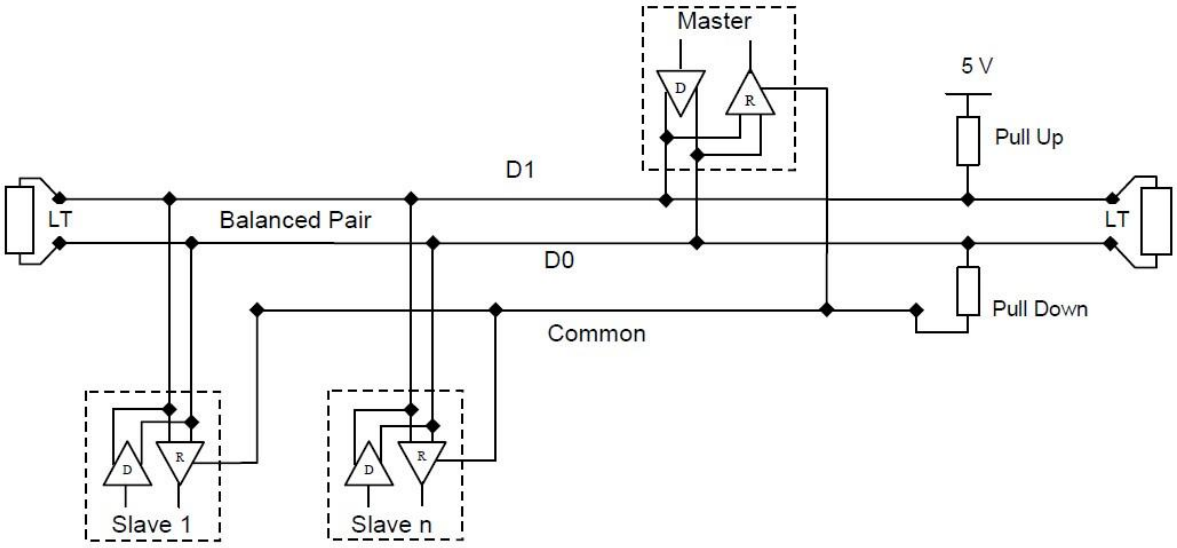

## Advantages of Modbus

- Open and license-free protocol
- Easy to implement and troubleshoot
- Supported by a wide range of manufacturers
- Reliable communication over long distances (RS-485)

## Typical Applications

- Industrial automation systems
- PLC-to-device communication
- Energy management systems
- Building automation
- Data acquisition and monitoring



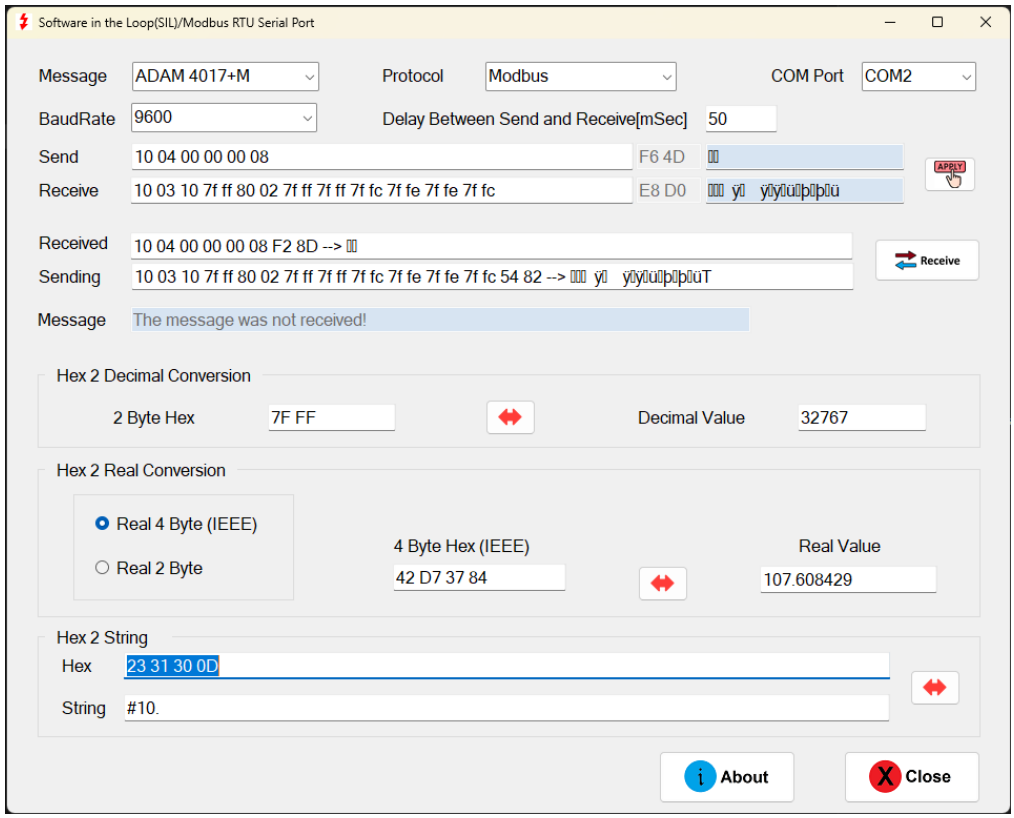




**General 2-Wire Topology**

## Software-In-the-Loop (SIL) - Serial / Modbus

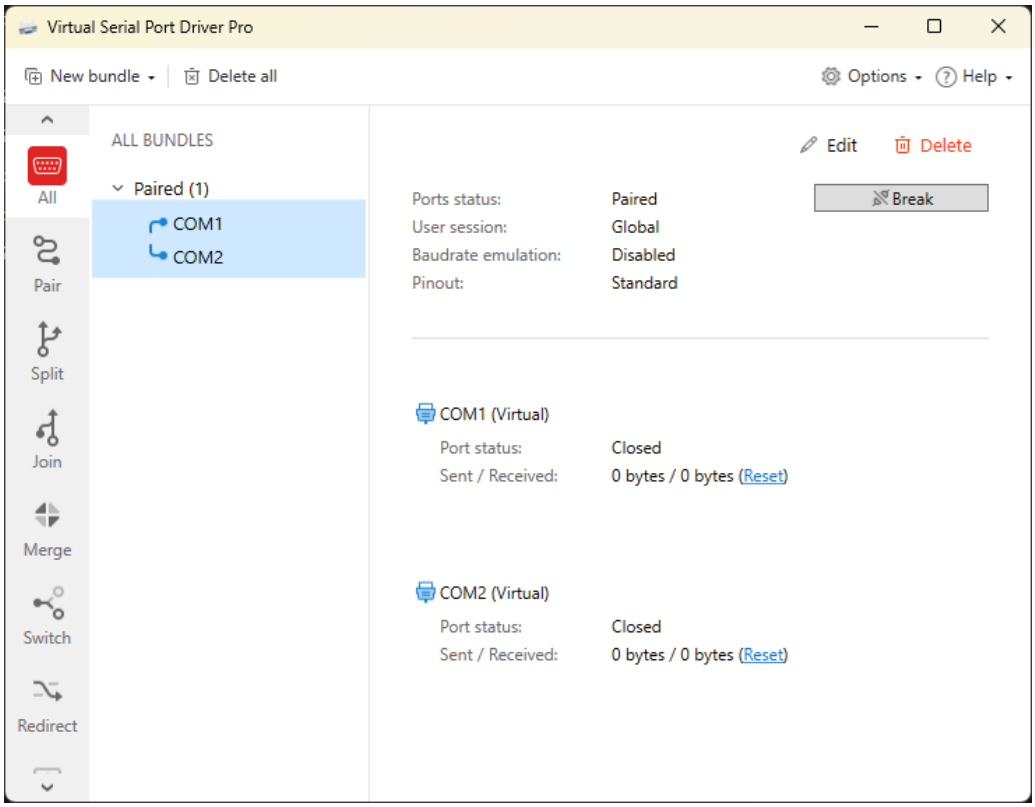
This software enables users to simulate a device connected through a serial port for a third-party application, acting on behalf of the real machine. It is particularly useful when the actual device is not available in the programming environment or when the machine requires significant power, space, or special operating conditions. The software allows the user to:

- Configure serial port settings
- Select or build the appropriate response to received commands
- Calculate the Modbus checksum (CRC) according to the Modbus RTU standard
- Wait a predefined number of milliseconds and then read the response
- Convert hexadecimal values to decimal
- Convert 4-byte hexadecimal values (IEEE floating-point format) to real numbers
- Convert 2-byte hexadecimal values to real numbers



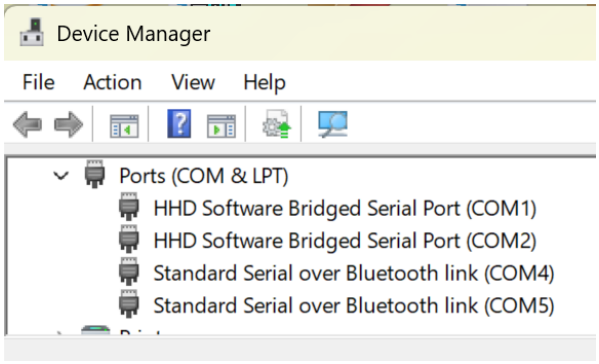
**How to add Serial Ports to Your Computer and Connect Them as a Loop**

When a device is connected to a computer through a serial port, it is often necessary to access real data for software development and testing. However, the actual device may not always be available at the development location, or the required operating conditions may be difficult to reproduce.
For example, the software may need to be tested under special conditions, such as very low temperatures or other environments that are not easily achievable in the development laboratory.
In such situations, a Software-In-the-Loop (SIL) approach can be used to simulate the behavior of the real device through software, allowing developers to test communication, data processing, and control logic without requiring the physical hardware.
A third-party tool such as “Virtual Serial Port Driver Pro” can be used to create a pair of connected virtual serial ports on the computer for simulation purposes.

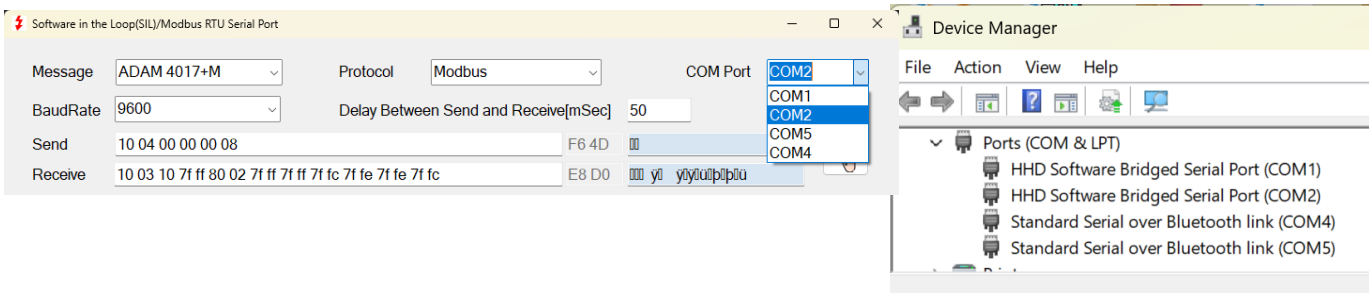


**How to Identify a Serial Port on Your Computer**

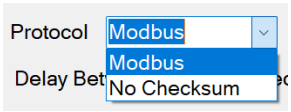
- Open the **Device Manager** application.
- Expand the **Ports (COM & LPT)** section.
- If a serial port is available, it will be listed there, as shown in the snapshot below.



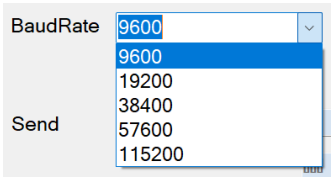

You can choose one of the existing ports as shown in the picture.




How to change the Protocol




How to change the Baud Rate




The latest settings are saved in the "**ModbusConverSIL.ini**" file and are restored the next time the software is opened.

**How to choose a message**

The operator can configure up to 10 messages in the "**MessageSIL.ini**" file, which contains both the message protocol definition and the corresponding display name used by the software. The package includes predefined messages for three Advantech devices: "**ADAM-4017**" for reading eight voltage or current channels, "**ADAM-4015**" for reading six **PT100** temperature sensors, and "**ADAM-4018**" for reading eight **thermocouple** temperature sensors. For each device, two lines must be defined in the file. The first line represents the message sent by the computer to the device. When the **SIL** software receives this message, it recognizes the command and prepares the corresponding response. In other words, the message sent from the computer must first be received by the SIL device in order to generate the appropriate response. The structure of the message is shown in the following sample of the "**MessageSIL.ini**" file.

10 04 00 00 00 08;Modbus;30;ADAM 4017+;Advantech Adam 4017+ Modbus protocol for A/D
10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc; We must add Modbus protocol CRC

13 04 00 00 00 06;Modbus;30;ADAM 4015;Advantech Adam 4015 Modbus protocol for PT100 temperature Sensor
13 03 0c 5f 21 5D 63 58 5A ff ff ff ff ff ff;We must add Modbus protocol CRC

16 04 00 00 00 08;Modbus;30;ADAM 4018;Advantech Adam 4018 Modbus protocol for thermocouple Sensor
16 03 10 5f 21 5D 63 58 5A ff ff ff ff ff ff ff ff;We must add Modbus protocol CRC

23 31 30 0D;No Checksum;30;ADAM 4017;Advantech Adam 4017 Advantech protocol for A/D
3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37 2E 34 37 31 32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 20 0D;
>+7.2111+7.2567+7.3125+7.1000+7.4712+7.2555+7.1234+7.5678 (cr)

23 31 33 0D;No Checksum;30;ADAM 4015;Advantech Adam 4015 Advantech protocol for PT100 temperature Sensor
3E 2B 32 32 2E 31 31 31 2B 32 33 2E 32 32 32 2B 32 34 2E 33 33 33 2B 32 35 2E 34 34 34 2B 32 36 2E 35 35 35 2B 32 37 2E 38 38 38 20 0D; >+22.111+23.222+24.333+25.444+26.555+27.888 (cr)

23 31 36 0D;No Checksum;30;ADAM 4018;Advantech Adam 4018 Advantech protocol for thermocouple Sensor
3E 2B 32 32 2E 31 31 31 2B 32 33 2E 32 32 32 2B 32 34 2E 33 33 33 2B 32 35 2E 34 34 34 2B 32 36 2E 35 35 35 2B 32 37 2E 38 38 38 2B 32 38 2E 39 39 39 2B 32 39 2E 31 31 31 20 0D;
>+22.111+23.222+24.333+25.444+26.555+27.888+28.999+29.111 (cr)

Each message is defined using two lines in the configuration file. Each part of a line is separated by a semicolon (;), as shown below.

In 1st line (Command Received by **SIL**):

- The **first part** defines the message that is received by the Software-In-the-Loop (SIL), when the corresponding command is sent from the external program. The message is written in **hexadecimal format**. Each ASCII character is represented by **two hexadecimal digits** ranging from 00 to FF. For example, 10 hex(**hexadecimal)** is equal to 16 in **decimal**. Each byte is separated by a space.
- The **second part** defines the **message protocol**. Two options are available: **"Modbus"** or **"No Checksum"**. If **"Modbus"** is selected, the checksum (CRC) is calculated and automatically appended to the message before transmission.
- The **third part** specifies the device response **delay** in milliseconds. After receiving the message, the device may require some time for internal processing before sending the response.
- The **fourth part** is the **name of the device** associated with the message.
- The **fifth part** is a **remark field**, used to provide additional information or comments.

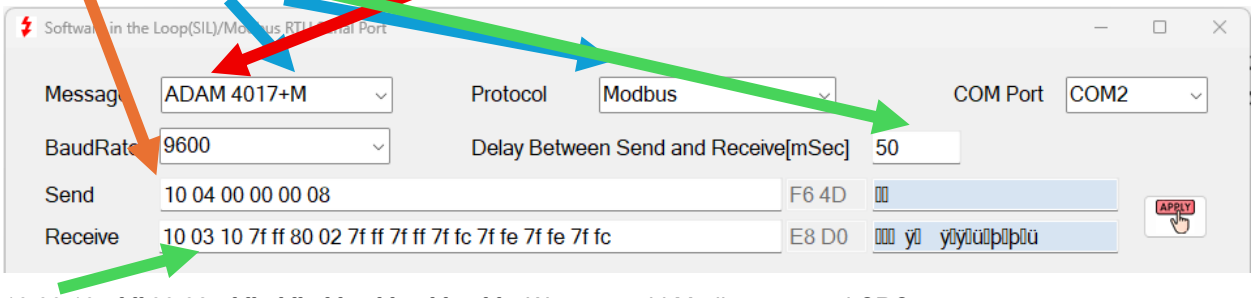
In 2nd line (Response Sent by **SIL**):

- The 2nd line defines the response message sent by the SIL software after the corresponding command has been received. The response is also written in **hexadecimal format**. Each ASCII character is represented by **two hexadecimal digits** ranging from 00 to FF. For example, 10 in **hexadecimal** is equal to 16 in **decimal**. Each byte is separated by a space. If the **message protocol** selects **"Modbus",** the checksum (CRC) is calculated and automatically appended to the message before transmission.

If **"Modbus"** is selected in the 2nd part, the letter **"M"** is appended to the end of the device name (in the 4th part), and if **"No Checksum"** is selected, an asterisk **("*")** is appended to the device name.

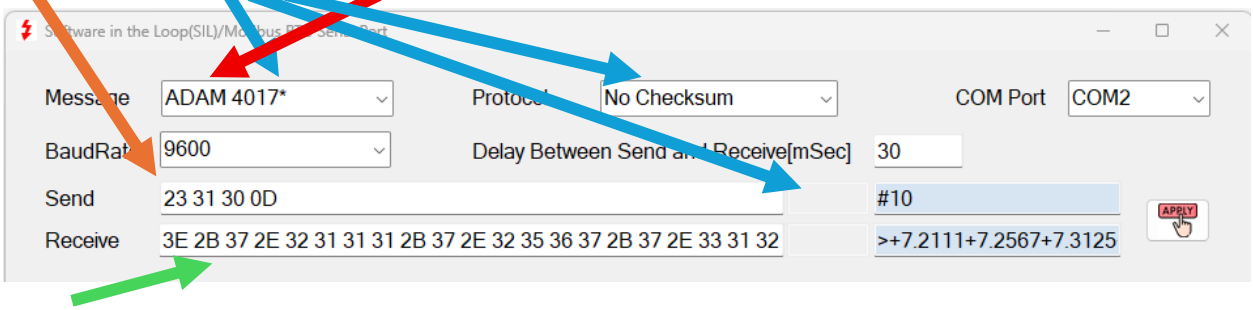
In our example, **Advantech** devices can also operate using a special Advantech message such as "**23 31 30**", which corresponds to the ASCII string "**#10**". In this case, the checksum is not applied.

10 04 00 00 00 08;Modbus;50;ADAM 4017+;Advantech Adam 4017+ Modbus protocol for A/D



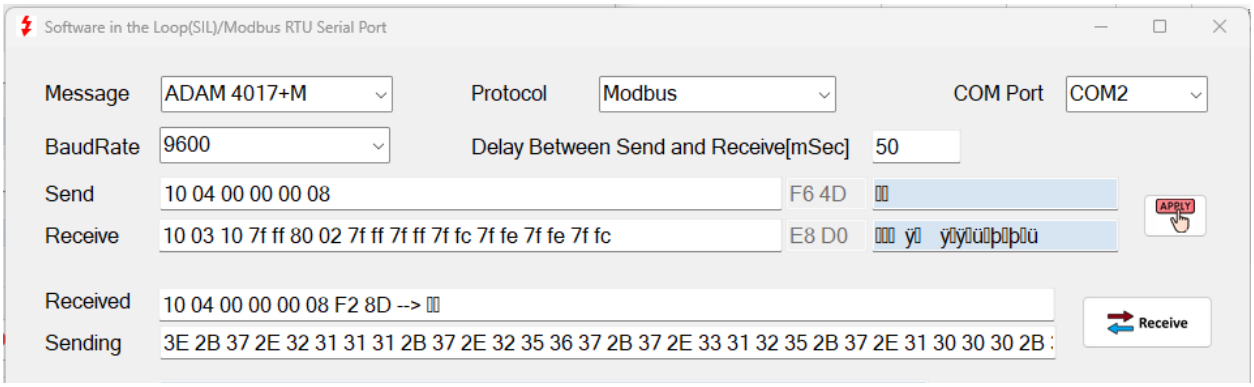

10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc; We must add Modbus protocol CRC

23 31 30 0D;No Checksum;30;ADAM 4017;Advantech Adam 4017 Advantech protocol for A/D




3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37 2E 34 37 31 32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 20 0D;
>+7.2111+7.2567+7.3125+7.1000+7.4712+7.2555+7.1234+7.5678 (cr)

Up to 10 predefined messages can be selected, along with one additional message that supports live editing.




The “Receive” button initiates **loop-based operation** and **hardware simulation** scenarios. When activated, the configuration icons are disabled, and the software waits to receive a message. The current status is displayed in the “**Message**” section.

All sent and received data are logged in the "**ModbusConverSIL.txt**" file.

Sample content of the "**ModbusConverSIL.txt**" file is shown below:

This text file was generated by ModbusConver.exe and developed by Babak Hajari. (Caution: Do not delete this line.)

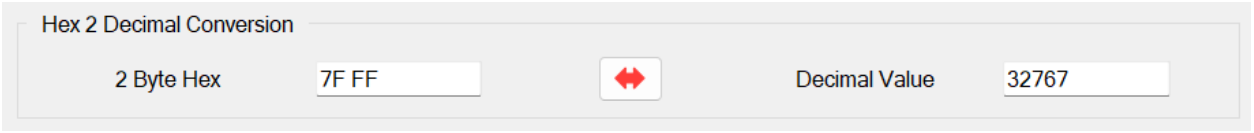
This text file was generated by ModbusConver.exe and developed by Babak Hajari. (Caution: Do not delete this line.)

2026/03/08 11:08
Message => Check CRC was OK. the data from
Send M => 10 03 10 7f ff 80 02 7f ff 7f ff 7f fc 7f fe 7f fe 7f fc 54 82
Received => 10 04 00 00 00 08 F2 8D
Send => 3E 2B 37 2E 32 31 31 31 2B 37 2E 32 35 36 37 2B 37 2E 33 31 32 35 2B 37 2E 31 30 30 30 2B 37 2E 34 37 31 32 2B 37 2E 32 35 35 35 2B 37 2E 31 32 33 34 2B 37 2E 35 36 37 38 20 0D 8C 40

# Calculation

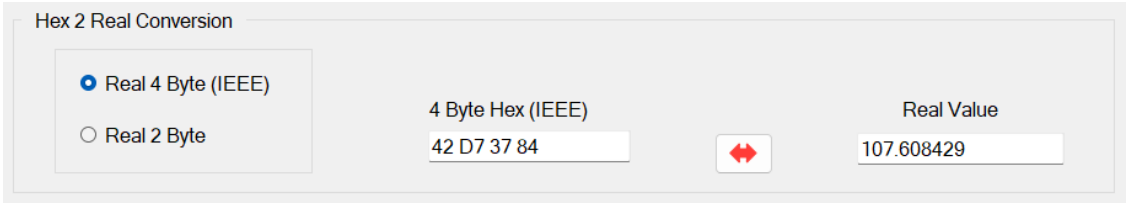
## Hex-to-Decimal Conversion

In this section, you can convert **2-byte hexadecimal values** to **decimal values**, and vice versa.

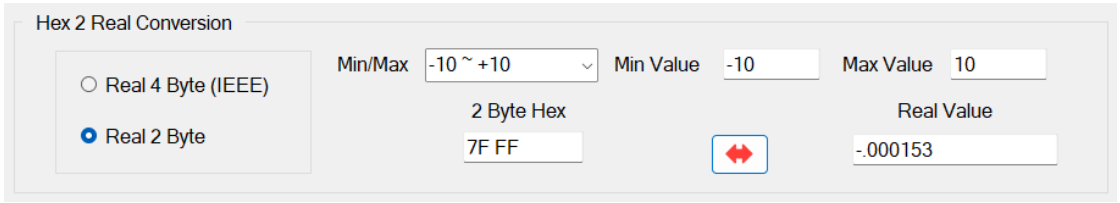



## Hex-to-Real Conversion

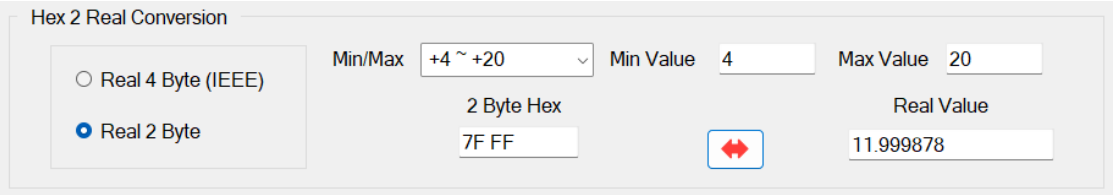
In this section, you can convert **4-byte hexadecimal values (in IEEE floating-point format) to real values** and vice versa.



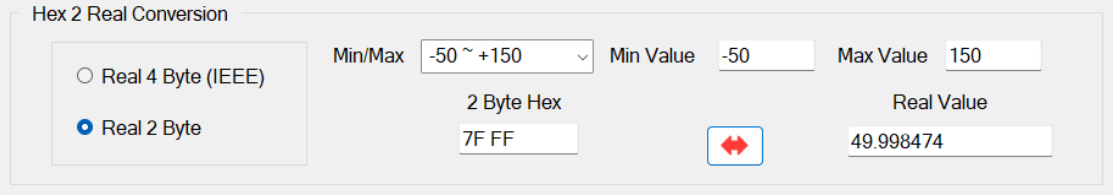

In this section, you can convert **2-byte hexadecimal values** into **real values** using defined **minimum and maximum limits**. For example, if the minimum value is **–10 [volts]** and the maximum value is **10 [volts]**, the corresponding real value is calculated based on the hexadecimal input.



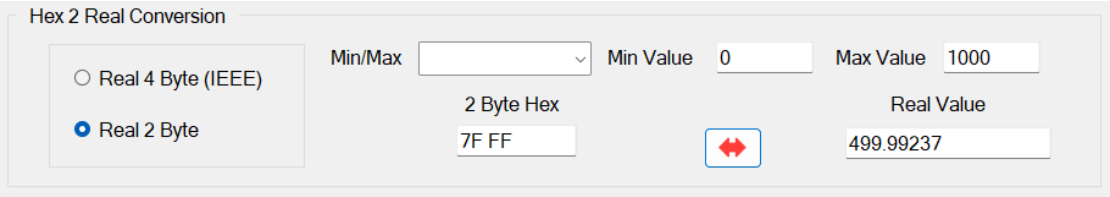
For example, if the minimum value is **4 [milli-amperes]** and the maximum value is **20 [milli-amperes]**, the corresponding real value is calculated based on the hexadecimal input.




For example, if the minimum value is **-50 [ºC]** and the maximum value is **150 [ºC]**, the corresponding real value is calculated based on the hexadecimal input.




If the minimum and maximum values are not listed among the predefined values, you can enter them directly in the Min and Max fields.




### Hex-to-String Conversion

In this section, you can convert **hexadecimal values to** a **string** and vice versa.

By pressing the "**About**" button, information about the package and its developer is displayed in a new window, as shown below.